

Lista de chequeo profesional para frontend de Dashboard

1. Arquitectura y estructura

- La aplicación tiene una arquitectura frontend modular y escalable.
- Los módulos están claramente separados.
- Existe una estructura consistente para páginas, componentes, servicios, stores y utilidades.
- Los componentes reutilizables están centralizados.
- La lógica de negocio no está mezclada innecesariamente con la presentación.
- Las llamadas a la API están centralizadas mediante servicios.
- Existe manejo global de estado cuando realmente es necesario.
- Las rutas están protegidas según autenticación y permisos.
- Existe una estrategia clara para manejar errores globales.
- Existe una estrategia clara para manejar estados de carga.

2. Autenticación y autorización

- Existe login seguro.
- Se controla correctamente la sesión del usuario.
- Se manejan expiración y renovación de tokens cuando corresponda.
- Las rutas privadas no pueden ser accedidas directamente sin autorización.
- El menú cambia según los permisos del usuario.
- Los botones y acciones sensibles respetan los permisos.
- No se confía únicamente en la ocultación de elementos del frontend.
- El backend continúa validando todos los permisos.
- Existe cierre de sesión correcto.
- Se evita almacenar información sensible innecesariamente en `localStorage`.

3. Layout principal del Dashboard

- Sidebar claramente organizado.
- Menú agrupado por módulos funcionales.

- Header con información del usuario.
- Breadcrumbs cuando sean útiles.
- Indicador claro de la sección actual.
- Sidebar responsive.
- Posibilidad de colapsar el sidebar.
- Diseño correcto para escritorio.
- Diseño correcto para tablet.
- Diseño correcto para móvil.
- No existen elementos que se desborden horizontalmente.

4. Navegación

- La navegación es predecible.
- Cada pantalla tiene una ubicación clara dentro de la aplicación.
- Los enlaces tienen nombres comprensibles.
- No existen enlaces muertos.
- No existen rutas duplicadas innecesariamente.
- Las rutas inexistentes muestran una página 404 útil.
- El usuario puede regresar fácilmente a la pantalla anterior.
- Las acciones importantes requieren confirmación cuando corresponda.

5. Captura de datos

- Los formularios tienen una estructura clara.
- Cada campo tiene una etiqueta visible.
- Los campos obligatorios están identificados.
- Se utilizan los tipos de input adecuados.
- Las opciones utilizan select, radio buttons o componentes apropiados.
- Los campos tienen valores predeterminados razonables.
- Se valida información antes de enviarla.
- Los errores aparecen junto al campo correspondiente.
- Los mensajes de error son comprensibles.

- Se evita perder información introducida accidentalmente.
- Los formularios largos están divididos por secciones.
- Existe indicador de progreso cuando una encuesta tiene varias etapas.
- Se puede guardar parcialmente cuando el proceso lo requiere.
- Se controla correctamente el envío duplicado.
- El botón de guardar muestra estado de procesamiento.
- Después de guardar se muestra una confirmación clara.

6. Encuestas

- La encuesta tiene un título y descripción claros.
- Las preguntas están organizadas lógicamente.
- Las preguntas obligatorias están identificadas.
- Se muestran instrucciones cuando son necesarias.
- Las opciones de respuesta son fáciles de seleccionar.
- Las preguntas pueden incorporar diferentes tipos de respuesta.
- Se soportan preguntas de texto.
- Se soportan preguntas numéricas.
- Se soportan preguntas de selección.
- Se soportan preguntas múltiples.
- Se soportan fechas.
- Se soportan escalas.
- Se puede mostrar progreso: **Pregunta 5 de 20.**
- Se evita que el usuario abandone accidentalmente una encuesta.
- Las respuestas quedan correctamente asociadas al encuestado.
- Se controla el envío de una encuesta ya completada.
- Existe una experiencia adecuada para encuestas largas.

7. Tablas de datos

- Las tablas tienen encabezados claros.
- Las columnas muestran información realmente relevante.

- Existe paginación.
- Existe búsqueda.
- Existe ordenamiento.
- Existen filtros.
- Los filtros pueden limpiarse fácilmente.
- Se muestra el número de resultados.
- Se muestra claramente cuando no existen registros.
- Se muestra estado de carga.
- Se maneja correctamente el error de consulta.
- Las tablas son responsive.
- Las acciones están agrupadas.
- Las acciones destructivas requieren confirmación.
- Las columnas pueden ocultarse cuando existen muchas variables.
- Se evita cargar miles de registros innecesariamente en el navegador.

8. Búsqueda y filtros

- La búsqueda tiene comportamiento consistente.
- Se diferencia búsqueda de filtros.
- Los filtros relevantes están disponibles.
- Los filtros pueden combinarse.
- Se puede limpiar todos los filtros.
- Se visualizan los filtros activos.
- Los filtros no generan solicitudes innecesarias.
- Se utiliza debounce en búsquedas apropiadas.
- El estado de los filtros se conserva cuando tiene sentido.

9. Estadísticas y KPIs

- El dashboard principal muestra los indicadores más importantes.
- Cada KPI tiene un nombre comprensible.
- Cada KPI tiene contexto.

- Las cifras tienen formato correcto.
- Se utilizan porcentajes cuando corresponde.
- Se utilizan cantidades absolutas cuando corresponde.
- Se muestran tendencias cuando aportan valor.
- Se pueden comparar períodos.
- Se pueden aplicar filtros a las estadísticas.
- Los gráficos representan correctamente los datos.
- No se utilizan gráficos únicamente por razones estéticas.
- Cada gráfico responde una pregunta concreta.
- Los gráficos tienen títulos claros.
- Las leyendas son comprensibles.
- Los valores importantes pueden consultarse sin depender exclusivamente del gráfico.
- Se muestran estados de carga.
- Se muestran estados sin datos.
- Se muestran errores de consulta.
- Se evita sobrecargar el dashboard con demasiados gráficos.

10. Visualización de datos

- Los gráficos tienen una jerarquía visual clara.
- Se utilizan gráficos de barras para comparaciones.
- Se utilizan gráficos de líneas para tendencias.
- Se utilizan gráficos circulares únicamente cuando realmente aportan valor.
- Se utilizan mapas solamente cuando la dimensión geográfica es relevante.
- Las escalas son correctas.
- No existen gráficos engañosos.
- Los tooltips muestran información adicional útil.
- Los gráficos funcionan en diferentes tamaños de pantalla.
- Existe alternativa tabular para datos importantes.

11. Estados de la interfaz

Cada pantalla debe contemplar explícitamente:

- Estado inicial.
- Estado de carga.
- Estado con datos.
- Estado sin datos.
- Estado de error.
- Estado de permisos insuficientes.
- Estado de sesión expirada.
- Estado de procesamiento.
- Estado de éxito.
- Estado de eliminación o actualización.

12. UX

- El usuario entiende qué puede hacer en cada pantalla.
- Las acciones principales son evidentes.
- Se evita utilizar demasiados botones.
- Los botones tienen verbos claros: Crear, Guardar, Editar, Eliminar.
- Los mensajes no utilizan lenguaje técnico innecesario.
- Las acciones importantes tienen feedback inmediato.
- No se utilizan alertas innecesarias.
- Los modales se utilizan únicamente cuando aportan valor.
- Los formularios no obligan al usuario a repetir información.
- Se minimiza la cantidad de clics para tareas frecuentes.
- Se mantienen patrones visuales consistentes.

13. Diseño visual

- Existe un sistema de diseño consistente.
- Se utilizan colores institucionales definidos.
- Existe una escala tipográfica consistente.

- Existe una escala de espaciado consistente.
- Los botones tienen estilos consistentes.
- Los formularios tienen estilos consistentes.
- Las tarjetas tienen estilos consistentes.
- Las tablas tienen estilos consistentes.
- Los iconos tienen un lenguaje visual consistente.
- Se evita utilizar demasiados colores.
- Existe suficiente contraste.
- La información importante tiene jerarquía visual.

14. Responsive Design

- Desktop.
- Laptop.
- Tablet.
- Smartphone.
- Formularios adaptables.
- Tablas adaptables.
- Gráficos adaptables.
- Sidebar adaptable.
- Modales adaptables.
- No existen botones demasiado pequeños en móvil.
- No existe scroll horizontal innecesario.

15. Accesibilidad

- Los elementos interactivos pueden utilizarse con teclado.
- Los campos tienen labels correctamente asociados.
- Existe foco visual.
- El contraste cumple estándares adecuados.
- Los iconos importantes tienen etiquetas.
- No se depende únicamente del color para transmitir información.

- Los mensajes de error son accesibles.
- Los componentes respetan semántica HTML.
- Los gráficos tienen información alternativa cuando es necesario.
- Se contempla navegación con lectores de pantalla.

16. Rendimiento

- Se utiliza lazy loading para módulos grandes.
- Las rutas pueden cargarse bajo demanda.
- Las imágenes están optimizadas.
- No se descargan recursos innecesarios.
- Las tablas utilizan paginación o virtualización cuando corresponde.
- Se evita realizar consultas API duplicadas.
- Se implementa caching donde aporta valor.
- Se utilizan debounce/throttle apropiadamente.
- Se evita renderizar componentes innecesariamente.
- El bundle está optimizado.
- El dashboard carga rápidamente incluso con grandes cantidades de datos.

17. Comunicación con API

- Existe un cliente HTTP centralizado.
- Las URLs de API utilizan variables de entorno.
- Los headers están centralizados.
- La autenticación está centralizada.
- Los errores HTTP tienen tratamiento uniforme.
- Se manejan correctamente 400.
- Se manejan correctamente 401.
- Se manejan correctamente 403.
- Se manejan correctamente 404.
- Se manejan correctamente 409.
- Se manejan correctamente 422.

- Se manejan correctamente 429.
- Se manejan correctamente errores 500.
- Se evita mostrar errores internos del backend directamente al usuario.
- Las solicitudes tienen estados `loading`, `success` y `error`.

18. Manejo de errores

- Existe manejo global de errores.
- Los errores inesperados no rompen toda la aplicación.
- Existe una pantalla de error cuando corresponde.
- Los mensajes son comprensibles.
- Se registra información técnica cuando es necesario.
- Se diferencia error de validación, autorización, red y servidor.
- Se permite reintentar operaciones recuperables.

19. Seguridad frontend

- No existen secretos dentro del código frontend.
- Las claves privadas nunca se incluyen en variables públicas.
- Las variables `VITE_*` no contienen secretos.
- Se evita almacenar tokens sensibles innecesariamente.
- Se controla XSS.
- Se evita insertar HTML sin sanitización.
- Se validan archivos antes de subirlos.
- Se controla tamaño y tipo de archivos.
- Las operaciones sensibles requieren autorización del backend.
- CORS está correctamente configurado en backend.

20. Importación y exportación

Si la aplicación maneja grandes cantidades de información:

- Exportación a CSV.
- Exportación a Excel cuando sea necesaria.
- Exportación de estadísticas.

- Importación de datos.
- Validación previa de archivos importados.
- Vista previa antes de importar.
- Informe de registros rechazados.
- Informe de errores de importación.
- Indicador de progreso para procesos largos.

21. Gestión de usuarios

- Listado de usuarios.
- Creación de usuarios.
- Edición.
- Activación/desactivación.
- Roles.
- Permisos.
- Estado del usuario.
- Último acceso cuando sea necesario.
- Búsqueda.
- Filtros.
- Paginación.

22. Auditoría

Para una aplicación de captura de información institucional:

- Se identifica quién creó un registro.
- Se identifica quién modificó un registro.
- Se registra cuándo ocurrió.
- Se registra qué operación se realizó.
- Las operaciones críticas muestran confirmación.
- El frontend permite consultar historial cuando corresponda.
- Se diferencia información actual de información histórica.

23. Notificaciones

- Mensajes de éxito.
- Mensajes de error.
- Advertencias.
- Información.
- Confirmaciones.
- Toasts consistentes.
- Las notificaciones no bloquean innecesariamente al usuario.
- Las notificaciones importantes permanecen visibles el tiempo suficiente.

24. Calidad del código

- ESLint configurado.
- Formateador configurado.
- Convenciones de nombres consistentes.
- Componentes pequeños y reutilizables.
- No existen componentes gigantes difíciles de mantener.
- No existen funciones innecesariamente complejas.
- No existen duplicaciones importantes.
- No existen imports innecesarios.
- No existen variables sin utilizar.
- No existen URLs de API escritas directamente en múltiples componentes.
- Los tipos/interfaces están correctamente definidos si se utiliza TypeScript.

25. Testing

- Pruebas de componentes críticos.
- Pruebas de formularios.
- Pruebas de validaciones.
- Pruebas de autenticación.
- Pruebas de permisos.
- Pruebas de navegación.

- Pruebas de operaciones CRUD críticas.
- Pruebas de encuestas.
- Pruebas de filtros.
- Pruebas de estadísticas.
- Pruebas de escenarios de error.
- Pruebas responsive cuando sean críticas.

26. Observabilidad

- Se pueden detectar errores frontend en producción.
- Se registran errores JavaScript importantes.
- Se pueden identificar fallos de API.
- Se conocen tiempos de carga.
- Se puede identificar qué pantalla genera errores.
- No se registran datos personales o sensibles innecesariamente.

27. Experiencia para administradores

- Dashboard general.
- Resumen de encuestas realizadas.
- Personas encuestadas.
- Encuestas pendientes.
- Encuestas completadas.
- Indicadores principales.
- Filtros por período.
- Filtros geográficos.
- Filtros por programa, oficina, grupo u otra dimensión.
- Estadísticas comparativas.
- Exportación de resultados.
- Acceso a información detallada desde los indicadores.

28. Dashboard profesional

La pantalla principal debería responder rápidamente:

- ¿Cuántos registros existen?
- ¿Cuántas personas han sido encuestadas?
- ¿Cuántas encuestas están pendientes?
- ¿Cuál es el porcentaje de cobertura?
- ¿Qué indicadores requieren atención?
- ¿Cómo ha evolucionado la situación?
- ¿Dónde se concentra el problema?
- ¿Qué grupos presentan mayor afectación?
- ¿Qué cambió respecto al período anterior?
- ¿Qué acciones requieren atención inmediata?

29. Arquitectura recomendada de navegación

Una estructura típica podría ser:

Dashboard

- Resumen general
- Indicadores
- Estadísticas

Encuestas

- Encuestas
- Nueva encuesta
- Aplicar encuesta
- Encuestas pendientes
- Historial

Personas

- Personas
- Buscar persona
- Detalle
- Historial

Seguimiento

- Variables

- Estados
- Evolución
- Alertas

Reportes

- Reportes
- Estadísticas
- Exportaciones

Administración

- Usuarios
- Roles
- Permisos
- Configuración

30. Criterio final de aceptación

Antes de considerar terminado el frontend:

- Un usuario nuevo puede entenderlo sin capacitación extensa.
- Un usuario puede realizar las tareas principales rápidamente.
- La aplicación funciona correctamente con datos reales.
- La aplicación funciona correctamente con miles de registros.
- Todos los estados de UI están contemplados.
- No existen errores visibles en consola.
- No existen errores críticos de UX.
- El diseño es consistente.
- El sistema es responsive.
- El sistema es accesible.
- Las operaciones críticas están protegidas.
- Los permisos están correctamente representados.
- Los indicadores muestran información confiable.
- Los gráficos permiten tomar decisiones.
- Las tablas permiten encontrar información rápidamente.

- Las encuestas pueden completarse sin fricción.
- Los errores pueden recuperarse sin perder información.
- El rendimiento es adecuado.
- Existe estrategia de testing.
- Existe estrategia de monitoreo en producción.

Prioridad recomendada

No todas las características tienen la misma prioridad.

P0 — Obligatorio

- Autenticación.
- Autorización.
- Navegación.
- Formularios.
- Validación.
- CRUD.
- Estados loading/error/empty/success.
- Tablas.
- Búsqueda.
- Paginación.
- Dashboard básico.
- Responsive.
- Seguridad.
- Manejo correcto de API.

P1 — Alta prioridad

- Filtros avanzados.
- KPIs.
- Gráficos.
- Exportación.
- Historial.

- Auditoría.
- Roles y permisos avanzados.
- Encuestas multipaso.
- Guardado parcial.
- Optimización de rendimiento.

P2 — Mejora

- Personalización del dashboard.
- Atajos de teclado.
- Temas.
- Configuración avanzada.
- Animaciones.
- Preferencias del usuario.
- Widgets configurables.

P3 — Opcional

- Funcionalidades experimentales.
- Animaciones avanzadas.
- Personalizaciones visuales avanzadas.
- Funciones de productividad no esenciales.